



República Bolivariana de Venezuela
Ministerio del Poder Popular para la Educación
Universidad Central de Venezuela - Facultad de Ciencias
Escuela de Computación - (6303) - Base de Datos



PROYECTO PLATAFORMA FANHUB | FASE II

Plataforma de la Creator Economy.
Suscripciones y Contenido Exclusivo.

Estudiantes: Mariangel Suárez | V-30.260.673
Mateo Gonzalez | V-29.981.261
Adrian Salazar | V-29.900.089

Preparador: Maximiliano B. | Profesores: Lic. Reynaldo T. y Msc. Christian L.

Caracas D.C., Venezuela.
10 de marzo de 2026

INTRODUCCIÓN

Llevar un diseño conceptual a la realidad no es simplemente transcribir cajas y rombos a sentencias `CREATE TABLE`. Es, más bien, el momento de la verdad; la etapa donde las ideas abstractas, validadas durante la Fase 1, chocan con las reglas estrictas y la rigidez de un motor de bases de datos. Para esta fase del proyecto FanHub, el reto cambió de perspectiva para construir un cimiento tecnológico capaz de soportar las exigencias. No quisimos abordar esta fase como una simple tarea de "tipear código". Decidimos tratar SQL Server como nuestro lienzo, donde cada restricción (`CHECK`), cada tipo de dato y cada llave foránea fue colocada con una intención clara: proteger la integridad de FanHub. Aprovechamos la automatización y los scripts iterativos con T-SQL (como los ciclos `WHILE`) para inyectar volumen a nuestro Data Seeding, permitiéndonos generar miles de registros coherentes que simularan el caos y la concurrencia de una plataforma real, en lugar de conformarnos con un puñado de datos estáticos insertados a mano.

En el camino, naturalmente, se presentaron desafíos lógicos. Por ejemplo, ¿cómo garantizamos que una suscripción y su primera factura nazcan juntas o no nazcan en absoluto? El momento clave de esta etapa fue comprender el poder del manejo de transacciones atómicas en nuestros Procedimientos Almacenados. Entendimos que la lógica de negocio no es un capricho del código, sino el verdadero guardián del modelo. Lo que presentamos a continuación no es solo un conjunto de archivos `.sql` comprimidos; es un ecosistema de datos funcional. Es el resultado de un equilibrio entre diseño estructural, lógica de programación y la simulación de un negocio real, listo para recibir su primer millón de usuarios.

1. MODELO FÍSICO DDL

La transición del modelo conceptual Entidad-Relación al esquema físico en SQL Server exige una rigurosidad analítica estricta, análoga a la definición de dominios en una función matemática. En esta fase, cada atributo dejó de ser una abstracción visual para convertirse en una variable con restricciones precisas de espacio e integridad lógica. Para garantizar la unicidad e identidad matemática de las entidades fuertes, se implementaron claves primarias con la propiedad `IDENTITY(1,1)`, generando una función inyectiva que asigna una secuencia monótona estrictamente creciente a cada registro nuevo. Esta abstracción fue fundamental para mantener una integridad referencial limpia y altamente indexable en las claves foráneas a lo largo de todo el modelo. En cuanto a las cadenas de caracteres, se realizó una diferenciación estricta en la asignación de memoria basándonos en la varianza de los datos. Para atributos con cardinalidad invariante, como los `ultimos_4_digitos` de los métodos de pago, se aplicó el tipo `CHAR(4)`, evitando así el overhead de cálculo de longitud que el motor requiere para los campos dinámicos. Por el contrario, atributos con longitud variable como los correos electrónicos o los títulos de las publicaciones se mapearon como `VARCHAR`, asignando memoria de forma dinámica para optimizar el almacenamiento en disco.

El manejo de los estados lógicos y los valores numéricos requirió un análisis igualmente minucioso. Dado que el motor de SQL Server carece de un tipo de dato booleano nativo, se implementó el tipo `BIT` para modelar todos los atributos de estado continuo, tales como las banderas `es_nsfw`, `es_publica` o `esta_activo`. Desde una perspectiva de teoría de conjuntos, esto restringe el dominio estrictamente a valores binarios, blindando la lógica de negocio y optimizando el almacenamiento a bajo nivel al empaquetar múltiples columnas en un solo byte.

Por su parte, para los dominios financieros (`precio_actual`, `sub_total`, `monto_total`), se descartó categóricamente el uso de tipos de coma flotante como `FLOAT` o `REAL`. Estos tipos basan su arquitectura en estándares de aproximación que introducen márgenes de error de redondeo, lo cual es inaceptable en un entorno transaccional. En su lugar, se optó por el tipo `DECIMAL(10,2)` para

garantizar una precisión aritmética exacta. Además, aplicando axiomas estrictos del negocio, este dominio financiero se acotó al conjunto de los números racionales no negativos mediante la restricción CHECK, bloqueando directamente a nivel de motor cualquier anomalía como precios o cobros bajo cero.

Finalmente, se determinó que la capa de datos debía asumir la responsabilidad de hacer cumplir reglas de negocio críticas de forma autónoma, sin depender exclusivamente del backend. La validación que exige que los usuarios sean mayores de 13 años se resolvió mediante una inecuación matemática a nivel de esquema, fijando el año 2013 como cota superior temporal mediante la restricción CHECK (`fecha_nacimiento <= '2013-01-01'`). Esta decisión actúa como un axioma inquebrantable que aborta automáticamente cualquier transacción de registro anómala. Del mismo modo, para aquellos atributos cuyo rango de valores posibles pertenece a un conjunto finito y predefinido, se evitó la creación de tablas satélites innecesarias aplicando la cláusula CHECK IN. De esta manera, los formatos de video y los estados de suscripción quedaron estrictamente limitados a sus valores válidos de negocio, operando como un filtro hermético que previene la contaminación de la base de datos y garantiza la consistencia estructural requerida para las consultas y reportes estadísticos

2. ESTRATEGIA DE DATA SEEDING

El requerimiento de poblar la base de datos con un volumen masivo y realista de información representó un desafío algorítmico significativo en la transición hacia el modelo físico. La directriz era inequívoca: prescindir por completo de datos estáticos carentes de sentido semántico ("lorem ipsum") y construir, en su lugar, un ecosistema lo suficientemente denso e interconectado para someter a prueba el rendimiento de las consultas analíticas complejas exigidas por el negocio. El punto de partida innegociable para esta fase fue la integración estricta del script maestro suministrado por la cátedra. La inserción obligatoria de estos catálogos oficiales, ejecutada mediante la habilitación temporal de `IDENTITY_INSERT`, garantizó que las entidades de Categoría y Tipo de Reacción sirvieran como un ancla de integridad referencial absoluta. Sobre este cimiento estandarizado, el enfoque de generación del resto del universo de FanHub debía ser escalable, reproducible y lógicamente entrelazado.

Para escalar la creación de registros hacia las cuotas restrictivas del proyecto —como los 250 usuarios, 800 publicaciones y más de 2.500 interacciones—, se descartó de plano la inserción manual o la importación de archivos planos. En su lugar, se adoptó un enfoque programático utilizando ciclos `WHILE` nativos de T-SQL y variables de iteración. Sin embargo, el verdadero reto no era iterar, sino garantizar que la asignación de claves foráneas y atributos categóricos no generara un ecosistema sesgado. Para resolver esto, se implementó aritmética modular sistemática como motor de distribución pseudoaleatoria. Por ejemplo, en la generación de la tabla `Publicacion`, la asignación del autor se definió bajo la expresión $(@p \% 25) + 1$; esta simple operación matemática garantizó que las 800 publicaciones se distribuyeran de manera perfectamente equitativa entre los 25 creadores del sistema. Esta misma lógica modular se aplicó para particionar los formatos multimedia: evaluando el residuo de $(@p \% 3)$, el algoritmo desvió exactamente un tercio de las publicaciones hacia la tabla `Video`, otro tercio hacia `Texto` y el remanente a `Imagen`, cumpliendo estrictamente con la regla de distribución equitativa impuesta por el modelo de negocio.

La aplicación de esta lógica de residuos (MOD) permitió también codificar reglas de negocio complejas directamente en el seeding. Para asegurar que el 30% del contenido inyectado mantuviera la bandera de exclusividad, se implementó una evaluación booleana `CASE WHEN (@p % 10) < 3`, forzando estadísticamente la proporción exacta sin importar el volumen de datos generado. No obstante, la distribución estructural debía ir acompañada de una estricta coherencia temporal, un factor crítico para el éxito de los reportes de rendimiento y retención (Churn). Esta variable se simuló dinámicamente mediante la función escalar `DATEADD`, restando intervalos matemáticamente vinculados al índice del ciclo respecto a la fecha actual del sistema (`GETDATE()`). Este enfoque permitió construir una línea de tiempo retrospectiva con sentido lógico irrefutable: las fechas de registro de los usuarios siempre preceden matemáticamente a sus fechas de suscripción, y estas, a su vez, son cronológicamente anteriores a la emisión de cualquier factura o interacción social.

Finalmente, la arquitectura del Data Seeding debió resolver dependencias existenciales y jerarquías recursivas para darle vitalidad al modelo. El ecosistema financiero requirió subrutinas de alta precisión; por cada una de las 500 suscripciones generadas, el script forzó la inserción simultánea de su respectiva factura. Para ello, se capturó el `precio_actual` del nivel en variables temporales, calculando el 16% de impuestos en tiempo de ejecución y concatenando variables para generar códigos de transacción (TRX) únicos y rastreables. En el ámbito social, la inserción de la tabla Comentario presentó un desafío topológico: simular conversaciones. Esto se resolvió dividiendo el bucle en dos fases; primero, la inserción de 900 comentarios raíz (con `idComentarioPadre` nulo), seguida de una subrutina que inyectó 100 comentarios apuntando mediante claves foráneas recursivas a los identificadores generados en la primera fase. Complementariamente, en las tablas transaccionales de intersección múltiple como `UsuarioReaccionPublicacion` y `PublicacionEtiqueta`, se blindó el algoritmo incorporando validaciones preventivas `IF NOT EXISTS`. Esta guarda lógica operó como un escudo contra violaciones de claves primarias compuestas, garantizando que la aleatoriedad modular no intentara registrar la misma etiqueta para un post o el mismo like de un usuario dos veces, asegurando que el script masivo se ejecutara de principio a fin sin una sola caída del motor de base de datos.

3. REGLAS DEL NEGOCIO (PROGRAMABILIDAD)

Una vez que ya teníamos las tablas listas y llenas de datos masivos, nos dimos cuenta de que la base de datos no podía ser solo un almacén inactivo de información. Teníamos que darle vida y hacer que respetara las reglas del negocio por sí sola. Para eso, utilizamos la programabilidad que ofrece T-SQL. Básicamente, lo que hicimos fue mudar gran parte de la lógica que normalmente programaríamos en el backend (o en el código de la aplicación) directamente al motor de la base de datos.

Para las operaciones que modificaban varias cosas al mismo tiempo, creamos Procedimientos Almacenados (SPs). El mejor ejemplo de esto fue el proceso de crear una nueva suscripción (`sp_crear_suscripcion`). Al principio pensamos que era solamente hacer un simple INSERT, pero la verdad es que si un usuario se suscribe, automáticamente también se le tiene que generar su primera factura para cobrarle. ¿Qué pasa si la suscripción se guarda bien pero la factura da error de conexión o algo parecido? Quedaría un dato a medias y la base de datos quedaría inconsistente. Para evitar ese dolor de cabeza, metimos todo el código dentro de un bloque de transacciones con BEGIN TRAN, COMMIT y ROLLBACK. Explicado de forma sencilla: si algo llega a fallar a mitad del camino, el sistema echa todo para atrás y es como si nada hubiera pasado. Lo mismo aplicamos con el procedimiento de publicar con etiquetas (`sp_publicar_con_etiquetas`), donde la lógica nos obligó a usar funciones para "picar" el texto de los tags separados por comas e insertarlos uno por uno en la base de datos.

Luego estaban los Triggers, que en nuestro diseño funcionan básicamente como unos vigilantes o "alcabalas" que revisan los datos en tiempo real, justo antes o después de que alguien intente modificarlos. Aquí implementamos dos principales para proteger el sistema. El primero es el de auditoría de precios (`tr_auditoria_precios`). La idea aquí es sencilla: si un creador se vuelve loco y quiere subir el precio de su nivel de suscripción de golpe en más de un 50%, el trigger detecta ese salto matemático, tumba la operación y lanza un mensaje de error. Es una medida de seguridad

antifraude. El segundo trigger es súper importante para el tema legal: la protección de menores (`tr_proteccion_menores`). Este trigger revisa las fechas de nacimiento antes de dejar que un usuario pague una suscripción; si se da cuenta de que el usuario tiene menos de 18 años y el creador al que quiere suscribirse tiene la etiqueta de contenido explícito (NSFW), la transacción se cancela de inmediato. Es decir, la misma base de datos se protege sola sin preguntarle a la aplicación.

Por último, para no tener que repetir el mismo código matemático a cada rato en nuestras consultas, armamos nuestras propias Funciones Definidas por el Usuario (UDF). Son literalmente como pequeñas calculadoras integradas en SQL. Por ejemplo, hicimos una para estandarizar el cálculo del impuesto (`fn_calcular_impuesto`) asumiendo el 16% de IVA, y otra para clasificar los ingresos en categorías como Diamante, Oro o Plata dependiendo del monto que facturen (`fn_clasificar_ingreso`). La que nos dio más trabajo fue la de la reputación (`fn_calcular_reputacion`), porque tuvimos que cruzar varias tablas para sumar el total de suscriptores, las reacciones recientes y el tiempo que lleva el creador en la plataforma, todo metido en una fórmula que no podía pasarse del tope de 100 puntos. Al tener esto guardado como una función, nos ahorramos muchos problemas a la hora de hacer los reportes finales, porque solo teníamos que llamarla en el `SELECT` y ella hacía todo el trabajo sucio.

4. CONSULTAS Y REPORTES

Llegar a este punto del proyecto fue como armar un rompecabezas gigante. Ya teníamos las 16 tablas creadas, los miles de registros insertados y las reglas de negocio funcionando de forma automática. Pero una base de datos no sirve de nada si no puedes extraer información útil de ella. El documento nos pedía resolver 14 consultas de analítica de negocio ("Reporting"), y rápidamente nos dimos cuenta de que un simple SELECT con un par de JOINS no iba a ser suficiente. Tuvimos que sentarnos a planificar cómo cruzar hasta seis o siete tablas al mismo tiempo sin que el servidor colapsara o nos devolviera filas duplicadas.

Uno de los mayores retos algorítmicos que enfrentamos aquí fue no poder usar simples funciones de agrupación como MAX() o SUM() para todo. Por ejemplo, en la Consulta 2, nos pedían sacar la publicación más viral por cada categoría basándonos en una fórmula matemática que sumaba reacciones y comentarios multiplicados por un peso específico. Para no enredar el código principal, descubrimos y utilizamos las Expresiones de Tabla Comunes (CTEs, usando la cláusula WITH). Esto nos permitió crear como una "tabla temporal en memoria" donde primero calculábamos todos los puntajes, y luego le aplicábamos una función de ventana llamada ROW_NUMBER() para numerar las publicaciones y poder filtrar fácilmente solo la número uno de cada categoría. Fue un salvavidas total. Esa misma técnica del WITH la aplicamos en la Consulta 8 para clasificar a los usuarios en "Gen Z", "Millennials" o "X" según su fecha de nacimiento antes de calcular sus promedios de gastos.

Otra cosa que nos obligó a pensar fuera de la caja fue el manejo de fechas y conversiones de tipos de datos. En la Consulta 4, que medía el "Churn" o promedio de retención de los usuarios que cancelaban, tuvimos que usar la función DATEDIFF para restar la fecha de inicio de la fecha de fin de cada suscripción y luego promediar esos días. Pero donde de verdad tuvimos que echar lápiz fue en la Consulta 5 (la del peso del contenido de Gaming). Ahí tuvimos que agarrar los segundos totales de los

videos, aplicar operaciones de módulo (%) para convertirlos a un formato legible de "Horas y Minutos" en formato texto, y al mismo tiempo usar un CASE WHEN grandísimo para multiplicar esos minutos por 0.5 GB, 0.1 GB o 0.05 GB dependiendo de si la resolución del video era 4K, 1080p o 720p.

Finalmente, esta fase de consultas fue el momento perfecto para poner a prueba las funciones (UDFs) que habíamos programado en el paso anterior. En lugar de escribir lógica repetitiva, simplemente llamábamos a `dbo.fn_clasificar_ingreso()` en la Consulta 1 para que nos etiquetara a los creadores como Diamante, Oro o Plata en pleno SELECT. Lo mismo hicimos en el Ranking de Creadores (Consulta 10) usando nuestra función de reputación. Y para rematar, aplicamos subconsultas con la cláusula EXISTS en la Consulta 7 (Intereses Cruzados) para encontrar a esos usuarios raros que gastan más de 140 dólares combinando suscripciones de Tecnología y Fitness, evitando así los temidos productos cartesianos que duplicaban los montos. El proyecto cierra con la Consulta 14, el reporte de nómina, donde se calcula en tiempo real ese 20% de comisión que se queda FanHub y el 80% neto que va para la cuenta bancaria del creador, demostrando que nuestra base de datos está totalmente lista para operar financieramente en el mundo real.

CONCLUSIÓN

Llegar al final de esta Fase 2 nos deja una lección invaluable como futuros profesionales de la computación: una base de datos no es un simple archivador pasivo de información, es el verdadero cerebro y guardián de cualquier plataforma. Si en la Fase 1 nuestro mayor reto fue abstraer las necesidades del cliente en cajas, rombos y diagramas de código, en esta etapa el desafío fue materializar esas ideas enfrentándonos a la rigidez matemática y las reglas estrictas de un motor como SQL Server. Durante este proceso, logramos transformar un diseño conceptual en un ecosistema físico y tangible de 16 tablas perfectamente estructuradas y normalizadas.

Comprendimos que el volumen importa, pasando de insertar registros a mano a desarrollar algoritmos de Data Seeding capaces de simular matemáticamente el caos y la concurrencia de miles de interacciones reales. Más importante aún, interiorizamos que la seguridad y las reglas de negocio no pueden depender exclusivamente de la aplicación frontend o backend; al implementar Procedimientos Almacenados con transacciones atómicas, Funciones de cálculo y Triggers de protección, logramos blindar la integridad de la información desde sus cimientos.

Hoy podemos decir con total propiedad que FanHub ha dejado de ser un simple proyecto académico en un papel. Ahora es un motor transaccional robusto, optimizado y probado, capaz de responder a consultas analíticas complejas de mercado. La estructura tecnológica está construido y la base de datos está lista para salir a producción.